Two Pointers-1

Agenda:

1. Pair Sum = K in sorted array.
2. Count of pairs with sum = K
3. Pair Difference = K
4. Container with most water

① Pair Sum = K in a Sorted Array

Find any pair of indices (i, j) such than $A[i] + A[j] = K$

logic: Array is Sorted → place one point i at start (smallest)

place j at end (largest)

- if sum = K → ans. found
- if sum > K → we need smaller sum, point j to left (j--)
- if sum < K → we need larger sum, move i to right (i++)

```
Public int [] PairSum (int [] A, int K) {
        int i = 0;
        int j = A.length-1;
        while (i < j){
            int sum = A[i] + A[j];
            if (sum == K) return new int [] { i, j };
            else if (sum > K) j--;
              else {i++};
        }
        return new int [] {-1, -1};
}
```

TC = O(N)
SC = O(1)

---

② Count of Pairs with sum = K

$A[i] + A[j] = K$

```
public long countPairs (int [] A , int K){
        Long count = 0;
        int i = 0 ; j = A.length-1;
        while ( i < j) {
            int Sum = A [i] + A [j];
            if (sum == K) {
                int x = A [i] ;
                int count i = 0;
                while (i < j && A[i] == x) {
                    count i++;
                    i++;
                }
```

Continue until the 2 pointers cross

Store current value at the left pointer

Count how many time x appears from index i and move i forward past all duplicates of x

```
int y = A[j];          → store current value at right pointer
int count j = 0;

while (i <= j && A[j] == y) {      → count how many times y appears starting from index j
    count j++;                       and move j backward past all duplicates of y
    j--;
}

if (x == y) {      → case when all element between i and j same (ex: 0, 2, 4, 4, 4, 4, 6, 8)
    long total = count i + count j;      → total no. of unique pairs
    count += (total * (total - 1))/2;      → nC2
}                                  → when x == y

else {
    count += (long) count i * count j;      → x != y
    }                                  ↳ every occurence of x can pair with every occurence of y.
}

else if (sum < k) { i++; }
else { j--; }
}                                  } same as previous ques

return count;
}
```

---

③ **Pair Difference = k** → Find Pair (i, j) such that $A[j] - A[i] = k$ in a sorted array where $k > 0$.

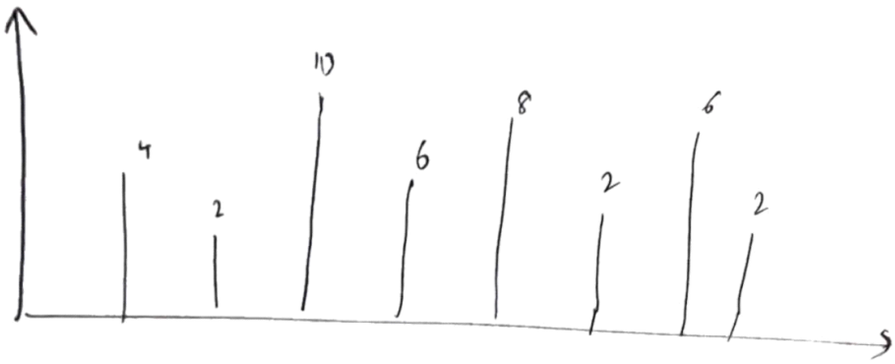
- $A[j] - A[i] < k$ → need a larger difference, increase A[j] by moving j to the right.
- $A[j] - A[i] > k$ → need a smaller diffence, increase A[i] which reduces the gap by moving i to the right.

```
int i = 0 ; j = 1
while (j < A.length) {
    int diff = A[j] - A[i];
    if (diff == k && i != j) return new int[]{i, j};
    else if (diff < k) j++;
    else {
        i++;
        if (i == j) j++;      → i == j → then diff = 0
    }
}
return new int[]{-1, -1};
}
```

(4) Container with Most Water

Given an integer array A[] where array elements, represents represents the height of the walls find any 2 walls that can form a container to store the maximum amount of water.
width between each wall is 1. [area = h × w]

Ex: A = {4, 2, 10, 6, 8, 2, 6, 2}



TC: O(n)
SC: O(1)

See notes for dry run

Sol.

```
Public int maxWater (int [] A) {
    int i = 0, j = A.length - 1;
    int maxArea = 0;
    while (i < j) {
        int height = Math.min (A[i], A[j]);
        int width = j - i;
        maxArea = Math.max (maxArea, height x width);
        if (A[i] < A[j]) i++;
        else j--;
    }
    return maxArea;
}
```

→ look for bigger i less than A[j]

look for bigger j less than A[i]